

# Shared Pages

## 1 Core Idea

### Definition

**Shared pages** are physical memory pages mapped into the address spaces of two or more processes.

Sharing avoids keeping multiple identical copies of the same program text or library code in RAM.

### Main Benefit

If several processes use the same read-only code, one physical copy can serve all of them.

## 2 What Can Be Shared

Not every page is equally easy to share.

| Page type               | Sharing difficulty                                           |
|-------------------------|--------------------------------------------------------------|
| <b>Program text</b>     | Easy to share because it is normally read-only.              |
| <b>Shared libraries</b> | Often sharable when mapped read-only or carefully protected. |
| <b>Data pages</b>       | Harder because processes may write different values.         |

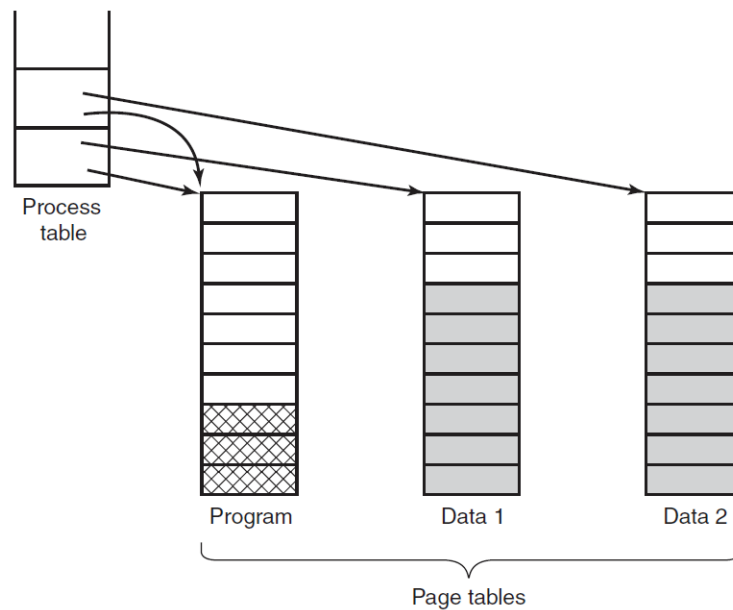
### Key Rule

Read-only pages are naturally sharable. Writable pages need extra mechanisms to avoid one process accidentally changing another process's private data.

## 3 Sharing with Separate I/D Spaces

If a system supports separate instruction and data spaces, sharing program code is straightforward.

- Processes share the same **I-space** page table for program text.
- Each process has its own **D-space** page table for data.



**Figure 3-25.** Two processes sharing the same program sharing its page tables.

### Figure Meaning

Two processes point to the same program page table, but each process points to a different data page table.

## 4 How the Process Table Uses Page Tables

In this design, page tables are separate data structures, not just fields inside the process table.

Each process table entry can contain:

| Pointer                           | Purpose                                      |
|-----------------------------------|----------------------------------------------|
| <b>I-space page-table pointer</b> | Locates the shared program-text mapping.     |
| <b>D-space page-table pointer</b> | Locates that process's private data mapping. |

When the scheduler runs a process, it uses these pointers to set up the MMU with the correct page tables.

## 5 Why Shared Pages Need Bookkeeping

Sharing creates ownership problems.

### Eviction Problem

If process *A* and process *B* share editor code, evicting all of *A*'s pages must not accidentally remove code pages still needed by *B*.

Similarly, when *A* exits, the OS must not free a shared page's disk space if *B* still uses it.

### Need for Metadata

The OS needs special data structures, such as reference counts or reverse mappings, to track which pages are shared and who still uses them.

Searching every process's page table to discover whether a page is shared is usually too expensive.

## 6 Sharing Without Separate I/D Spaces

Sharing is still possible without separate I-space and D-space, but the mechanism is more complicated.

| Case        | Issue                                                                               |
|-------------|-------------------------------------------------------------------------------------|
| Shared code | The OS must map the same physical code pages into multiple processes.               |
| Shared data | Writes must be controlled so that private data does not become accidentally shared. |

## 7 Copy on Write

### Definition

**Copy on write** lets processes share data pages until one process tries to modify a shared page.

This is commonly used after the UNIX `fork` system call.

## 8 Copy-on-Write Flow

After `fork`:

1. Parent and child get separate page tables.
2. Both page tables point to the same physical pages.
3. Shared data pages are marked read-only in both processes.
4. If both only read, no copying occurs.
5. If either process writes to a shared page, a protection fault traps to the OS.
6. The OS copies that page.
7. Each process gets its own private copy.
8. Both copies can then be marked read/write.

### Why Copy on Write Is Efficient

Only pages that are actually modified are copied. Pages that are never written, including program text pages, remain shared.

## 9 Copy-on-Write Example

### Scenario

Process  $P$  calls `fork`, creating child process  $C$ . Both initially share data page  $X$ , marked read-only.

| Event           | Result                                                                                |
|-----------------|---------------------------------------------------------------------------------------|
| $P$ reads $X$   | Allowed. Page remains shared.                                                         |
| $C$ reads $X$   | Allowed. Page remains shared.                                                         |
| $P$ writes $X$  | Trap occurs because $X$ is read-only.                                                 |
| OS handles trap | OS copies $X$ , gives $P$ a private writable copy, and leaves $C$ using the old copy. |

## 10 Summary of Sharing Techniques

| Technique                  | Purpose                                                   |
|----------------------------|-----------------------------------------------------------|
| Shared read-only code      | Avoid duplicate copies of program text or libraries.      |
| Shared I-space page table  | Let multiple processes use the same instruction mappings. |
| Private D-space page table | Keep each process's writable data separate.               |
| Reference tracking         | Prevent freeing shared pages still in use.                |
| Copy on write              | Delay copying data pages until a write actually happens.  |

## 11 Final Summary

### One-Box Summary

Shared pages reduce duplicate memory use.  
Read-only code is easy to share; writable data is harder.  
Separate I/D spaces make code sharing simple.  
The OS must track shared pages to avoid freeing them too early.  
Copy on write shares data until a process modifies it.